NATIONAL INSTITUTE OF TECHNOLOGY, WARANGAL

Department of Mechanical Engineering

PROJECT REPORT

ME355: Mechatronics

Ball Balancing Robot

Under the guidance of **Prof Maroju Naresh Kumar**

Submitted by:

Dhoma Vivekvardhan Reddy	23MEB0B08
Chinmay Manoj Patil	23MEB0B09
Rudransh Singh	23MEB0B10
Syed Nazeeruddin	23MEB0B12
Mohammad Irfan	23MEB0B13
Aritra Mandal	23MEB0B14

Abstract

The ball balancing system is a control engineering application designed to stabilize a ball on a flat platform by dynamically adjusting the platform's orientation. The system continuously detects the position of the ball and uses a feedback control mechanism to maintain the ball at a desired location, typically the center of the platform. In this project, a microcontroller-based control system is implemented to monitor the ball position and control the movement of the platform using motors.

A camera or sensor module is used to track the position of the ball in real time. The detected position is processed by the controller, which calculates the positional error relative to a predefined setpoint. A Proportional–Integral–Derivative (PID) control algorithm is applied to minimize this error by adjusting the tilt angles of the platform. The PID controller continuously generates corrective signals that drive the motors to stabilize the ball.

The system demonstrates the practical application of closed-loop control, real-time sensing, and mechatronic integration. This project provides a useful platform for understanding feedback control systems, robotics, and dynamic system stabilization. The results show that with proper PID tuning, the platform can effectively maintain the ball near the desired position with minimal oscillations and improved stability.

Introduction

A ball balancing platform is a robotic control system designed to keep a ball stable on a flat surface by dynamically adjusting the orientation of the platform. The system continuously monitors the position of the ball and tilts the platform in such a way that the ball moves toward a desired position, usually the center. This type of system represents a classic control engineering problem because the ball is inherently unstable and requires continuous feedback control to maintain equilibrium.

In this project, a robotic platform is used to balance a ball on a square plate. The system uses sensors or vision modules to detect the ball position and motors to tilt the platform accordingly. A feedback control algorithm processes the position error and determines the appropriate movement of the platform.

The control strategy implemented in the system is the **Proportional–Integral–Derivative (PID) controller**, which calculates the difference between the desired position (setpoint) and the actual position of the ball and adjusts the platform angle to minimize this error. The controller continuously updates motor commands to stabilize the ball and prevent it from rolling off the platform.

Ball balancing systems are widely used as educational platforms for studying robotics, control systems, feedback loops, and mechatronics integration.

Objectives

The main objectives of the ball balancing project are:

1. To design and develop a robotic platform capable of balancing a ball on a flat surface.

2. To implement a closed-loop control system for maintaining the ball at a desired position.

3. To detect the real-time position of the ball using sensors or a vision system.
4. To control the orientation of the platform using motors or servo mechanisms.
5. To implement and tune a PID controller for stable and accurate ball balancing.
6. To analyze system performance and evaluate the stability of the control algorithm.

Hardware Components & their functions:

The system is built using the following core components:

S .No	Component	Quantity	Category
1	4-Wire Resistive Touch Plate	1	Sensor
2	MG996R Servo Motor HIGH TORQUE	3	Actuator
3	Arduino Uno	1	Microcontroller
4	Buck Converter	1	Power
5	11.1v 2200Mah 3s Li-Po Battery	1	Power

1. 4-Wire Resistive Touch Plate:

Function:

The 4-wire resistive touch plate is used to **detect the position of the ball on the platform**. It consists of two conductive layers separated by a small gap. When the ball presses the surface, the layers make contact and create a voltage change that corresponds to the **X and Y coordinates** of the touch point.The microcontroller reads these voltage values and determines the exact position of the ball on the platform. This position data acts as the **feedback signal** for the control system, allowing the controller to determine how the platform should tilt to bring the ball back to the desired position.

2. MG996R Servo Motor HIGH TORQUE

Function:

The MG996R servo motors are responsible for **tilting the platform in different directions**. These high-torque motors rotate to specific angles when commanded by the microcontroller through PWM signals.

In the ball balancing system:

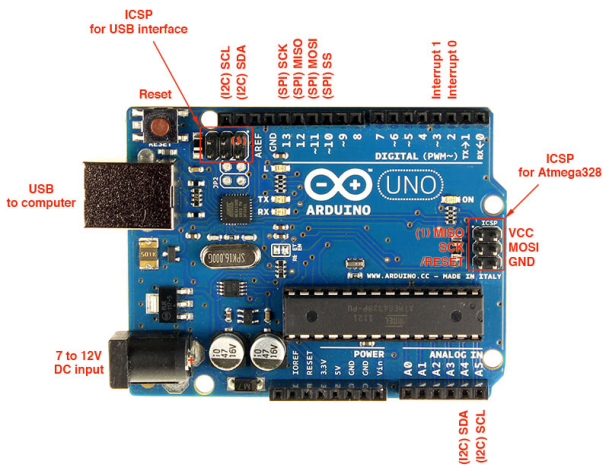
- The servos adjust the **tilt angle of the platform**
- This tilt causes the ball to roll due to gravity
- The system uses **three servos** to control the platform orientation precisely

By continuously adjusting the servo angles, the platform keeps correcting the ball's position and stabilizes it near the center.

3. Arduino Uno

Function:

The Arduino Uno acts as the **main control unit** of the system. It processes input signals from the touch plate and executes the control algorithm. Its main tasks include:



- Reading ball position data from the sensor
- Calculating the **error between the desired and actual position**
- Executing the **PID control algorithm**
- Generating PWM signals to control the servo motors

Based on these calculations, the Arduino sends commands to the servo motors to adjust the platform angle and stabilize the

ball.

4. Buck Converter (Voltage Regulator)

Function:

The buck converter is used to **step down the battery voltage to a suitable level** for the electronics. Since the battery provides **11.1 V**, but components like the Arduino and servos require lower voltages, the buck converter reduces the voltage safely. Its functions include:

- Converting **high battery voltage to regulated output voltage**
- Protecting electronic components from over-voltage
- Providing stable power to the microcontroller and sensors

This ensures reliable operation of the entire system.

5. 11.1v 2200Mah 3s Li-Po Battery

Function:

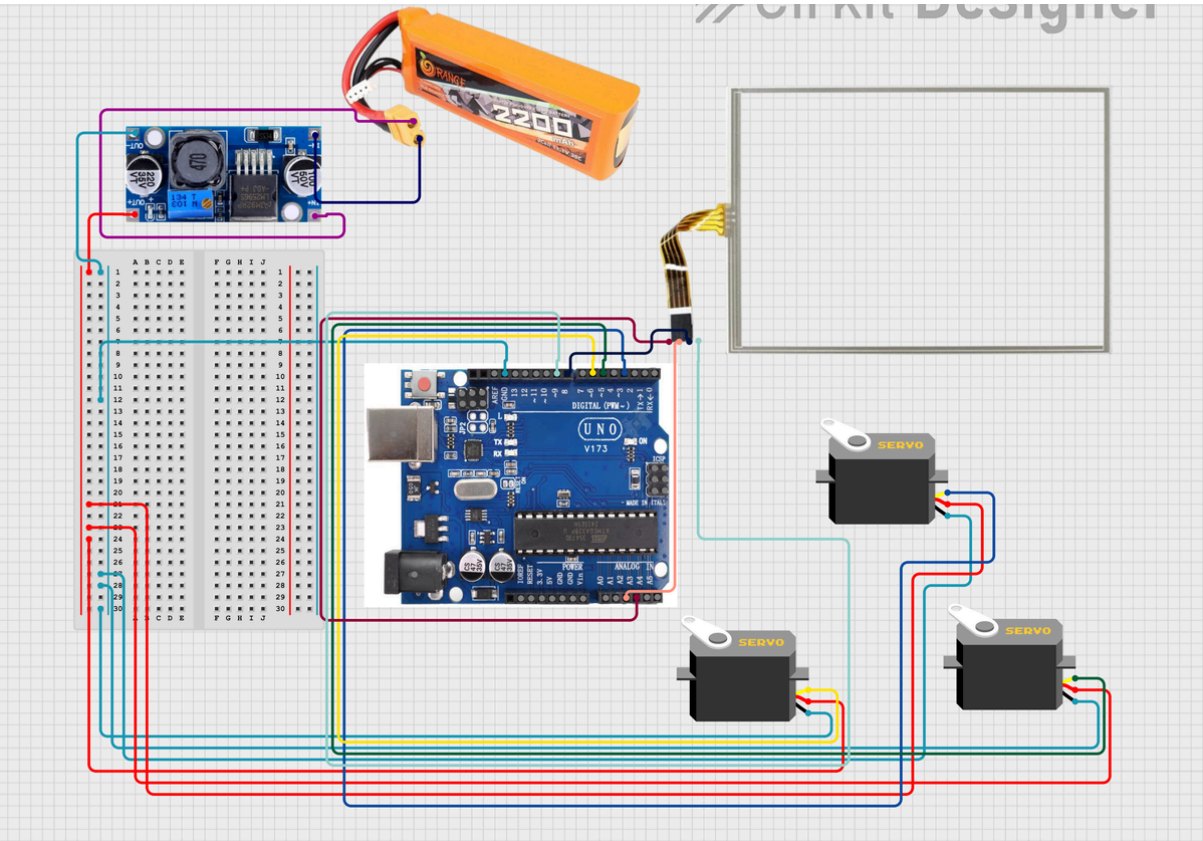
The Li-Po battery acts as the **main power supply for the entire system**.

It provides electrical energy to:

- Servo motors
- Arduino microcontroller
- Sensor system
- Voltage regulator

The **11.1 V (3S configuration)** ensures sufficient power for the high-torque servo motors, while the **2200 mAh capacity** allows the system to operate for extended periods without external power.

Circuit Diagram:



Code:

```
#include <Servo.h>

#include <TouchScreen.h>

#include <math.h>

#define XP 9

#define YP A2

#define XM A3

#define YM 8

#define TS_OHMS 765

#define MIN_PRESSURE 0

 if (p.z > MIN_PRESSURE && p.x > 100 && p.y > 100) {

   detected = true;

   for (int i = 0; i < 2; i++) {

     errorPrev[i] = error[i];

     error[i] = (i == 0) * (Xoffset - p.x - setpointX)

              + (i == 1) * (Yoffset - p.y - setpointY);

     integr[i] += error[i] + errorPrev[i];

     deriv[i] = error[i] - errorPrev[i];

     deriv[i] = (isnan(deriv[i]) || isinf(deriv[i])) ? 0 : deriv[i];
```

```
TouchScreen ts = TouchScreen(XP, YP, XM, YM, TS_OHMS);

Servo servoA;

Servo servoB;

Servo servoC;

#define PIN_A 3

#define PIN_B 5

#define PIN_C 6

#define HOME_ANGLE 105

#define SERVO_MIN  60

#define SERVO_MAX  160

int basec = 144;

int basea = 72;

int baseb = 103;

double Xoffset = 515;

double Yoffset = 500;

double kp_y = 0.0375;   // increase if reacting too slow


double ki_y = 0.0015;    // add last, prevents drift

double kd_y = 1.45;    // add after kp works, prevents overshoot

double kp_x = 0.061;   // increase if reacting too slow

double ki_x = 0.0009;    // add last, prevents drift

double kd_x = 1.5;

double error[2]     = {0, 0};

double errorPrev[2] = {0, 0};

double integr[2]    = {0, 0};

double deriv[2]     = {0, 0};

double out[2]       = {0, 0};

bool detected = false;
```

```
      out[0] = kp_x * error[0] + ki_x * integr[0] + kd_x * deriv[0];



  out[1] = kp_y * error[1] + ki_y * integr[1] + kd_y * deriv[1];

      out[0] = constrain(out[0], -12.0, 12.0);  // ±20 degrees max tilt

      out[1] = constrain(out[1], -12.0, 12.0);  // ±20 degrees max tilt

    }

    Serial.print("X OUT="); Serial.print(out[0]);

    Serial.print(" Y OUT="); Serial.print(out[1]);

    Serial.print(" Ball X="); Serial.print(p.x);

    Serial.print(" Ball Y="); Serial.println(p.y);

  } else {

    delay(10);

    TSPoint p2 = ts.getPoint();

    if (p2.z <= MIN_PRESSURE || p2.x <= 100 || p2.y <= 100) {

      detected = false;

      integr[0] = 0;

      integr[1] = 0;

    }

  }

  timeI = millis();

  while (millis() - timeI < 10) {

    if (detected) {

      moveTo(-out[0], out[1]);

    } else {

      moveTo(0, 0);

    }
```

```
long timel;

void moveTo(double tiltX, double tiltY) {

  double angleA = basea - tiltY;

  double angleB = baseb + 0.5 * tiltY + 0.866 * tiltX;

  double angleC = basec + 0.5 * tiltY - 0.866 * tiltX;

  int a = constrain((int)round(angleA), SERVO_MIN,
SERVO_MAX);

  int b = constrain((int)round(angleB), SERVO_MIN,
SERVO_MAX);

  int c = constrain((int)round(angleC), SERVO_MIN,
SERVO_MAX);

  servoA.write(a);

  servoB.write(b);

  servoC.write(c);

}

void PID(double setpointX, double setpointY) {

  TSPoint p = ts.getPoint();

  }

}

void setup() {

  Serial.begin(115200);

  servoA.attach(PIN_A);

  servoB.attach(PIN_B);

  servoC.attach(PIN_C);

  servoA.write(basea);

  servoB.write(baseb);

  servoC.write(basec);

  delay(1000);

  Serial.println("Platform ready!");

}

void loop() {

  PID(0, 0);

}
```

System Architecture

The ball balancing system follows a **closed-loop control architecture**. The architecture consists of sensing, processing, and actuation stages.

1. **Sensing Stage**
   The camera or sensor detects the ball's position on the platform and determines its coordinates in the X and Y directions.
2. **Processing Stage**
   The detected coordinates are sent to the microcontroller. The microcontroller calculates the difference between the desired position (setpoint) and the actual ball position.
3. **Control Algorithm**
   The PID controller processes the error and calculates the required platform orientation needed to correct the ball position.
4. **Actuation Stage**
   The calculated control signal is sent to the motors. The motors tilt the platform so that gravity causes the ball to roll toward the desired location.
5. **Feedback Loop**
   The system continuously repeats this process, updating the ball position and adjusting the platform until the ball remains stable at the setpoint.

This continuous feedback mechanism ensures real-time correction and stable ball balancing.

Control Methodology (PID)

The ball balancing system uses a **PID (Proportional–Integral–Derivative) controller** to maintain the ball at the desired position.

The controller calculates the error between the desired position and the actual ball position:

$$Error = Setpoint - Current\ Position$$

The PID controller consists of three components:

1. Proportional Control (P)

The proportional term produces an output proportional to the current error.

$$P = K_p \times error$$

It moves the platform in proportion to how far the ball is from the target position.

2. Integral Control (I)

The integral term accumulates the error over time.

$$I = K_i \int error\, dt$$

This component helps eliminate steady-state error and ensures that the ball eventually settles exactly at the desired position.

3. Derivative Control (D)

The derivative term predicts the future behavior of the system based on the rate of change of error.

$$D = K_d \frac{d(error)}{dt}$$

It slows down the motion of the ball and reduces oscillations.

Final PID Output

The final control signal applied to the motors is:

$$Output = P + I + D$$

This output determines the required tilt of the platform. The system continuously adjusts the platform angle so that the ball moves toward the center, stabilizes, and

remains balanced.

Separate PID controllers are often implemented for the **X and Y directions** to control the platform tilt in two axes.

Results and Output

The ball balancing platform was assembled and tested under different operating conditions.

During experimentation:

- The system successfully detected the ball position in real time.
- Servo motors adjusted the platform orientation according to the PID output.
- After proper tuning of the PID parameters, the system stabilized the ball near the center with minimal oscillation.

The system demonstrated reliable balancing performance and consistent response to disturbances.

Advantages

1. Demonstrates practical implementation of feedback control systems.
2. Useful educational platform for studying PID controllers.
3. Combines mechanical design, electronics, and programming.
4. Provides real-time control system experimentation.
5. Scalable for advanced robotics and control research.

Future Improvements

1. Implementation of advanced control methods such as adaptive or fuzzy control.
2. Improved sensing accuracy using machine vision algorithms.
3. Integration of wireless communication for remote monitoring.
4. Addition of more degrees of freedom for enhanced control.
5. Use of faster processors for real-time image processing.

Conclusion

The Ball Balancing Platform developed in this project demonstrates the practical application of feedback control principles in a dynamic system. The integration of sensors, actuators, and a microcontroller enables real-time monitoring and control of the ball position.

The PID controller successfully stabilizes the ball on the platform by continuously adjusting the tilt angles. The project provides valuable insight into control systems, robotics, and embedded system design.

Overall, the system proves to be an effective educational tool for understanding the challenges associated with stabilizing unstable systems and implementing real-time control algorithms.

References

- Arduino Documentation — Arduino Uno Microcontroller
- PID Control Theory — Control Systems Engineering Textbooks
- Ball Balancing Platform Project Tutorials and Open Source Resources
- Embedded Systems Programming using Arduino IDE

Contributions

CAD model	Nazeer and Chinmay
3d print	Vivek, Irfan
Assembly	Nazeer , Chinmay ,Irfan
Circuit and Coding	Aritra
PID tuning	Irfan ,Vivek, Rudransh
Documentation	Vivek
Video Editing	Irfan
Coordination	Irfan and Vivek
3d printing support	Mani Kumar